National University

Of Computer & Emerging Sciences Chiniot- Faisalabad Campus

AI 2002 – Artificial Intelligence (Spring 2026)

Assignment 4

Topics Covered: Constraint Specification Problems (CSP)	**Submission Deadline:** Wed – *April 15, 2026 by23.00 sharp*

Submission Guidelines:

- **Solo Assignment:**
- You may use internet/chatgpt for help but **do not submit "copy-paste" answers**.
- Submit a PDF file containing your responses on the Google Classroom.
- Submit the code file for Q2. Zip all the submissions.

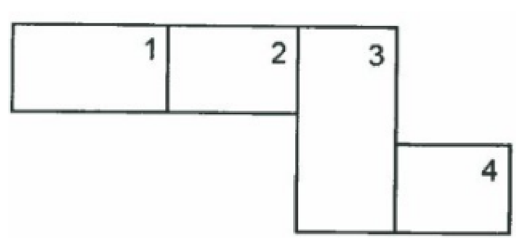
Question 1

The Zoo in Killian Court

In honor of MIT 150, MIT has decided to open a new zoo in Killian Court. They have obtained seven animals and built four enclosures. Because there are more animals than enclosures, some animals have to be in the same enclosures as others. However, the animals are very picky about who they live with. The MIT administration is having trouble assigning animals to enclosures, just as they often have trouble assigning students to residences. As you have taken 6.034, they have asked you to plan where each animal goes.

The animals chosen are a LION, ANTELOPE, HYENA, EVIL LION, HORNBILL, MEERKAT, and BOAR.

They have given you the plans of the zoo layout.



Each numbered area is a zoo enclosure. Multiple animals can go into the same enclosure, and not all enclosures have to be filled. Each animal has restrictions about where it can be placed.

The LION and the EVIL LION hate each other, and do not want to be in the same enclosure.

The MEERKAT and BOAR are best friends, and have to be in the same enclosure.

The HYENA smells bad. Only the EVIL LION will share his enclosure. 4. The EVIL LION wants to eat the MEERKAT, BOAR, and HORNBILL.

The LION and the EVIL LION want to eat the ANTELOPE so badly that the ANTELOPE cannot be in either the same enclosure or in an enclosure adjacent to the LION or EVIL LION.

The LION annoys the HORNBILL, so the HORNBILL doesn't want to be in the LION's enclosure.

The LION is king, so he wants to be in enclosure 1.

Part1

1) Define the CSP: (4 marks)

List all variables (animals)

List the initial reduced domains provided below

2) Draw the Constraint Graph: (6 marks)

Represent each variable (animal) as a node

Draw an edge between any two variables that have a direct constraint between them (binary constraints)

For constraints involving more than two variables (e.g., MEERKAT BOAR),represent them appropriately

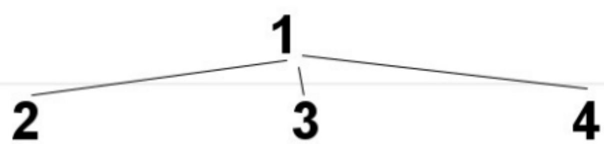
Part 2:

1) find one solution using depth first search with forward checking and propagation through domains reduced by any number of values (propagation through reduced domains.) Show your work by filling out the domain worksheet.Break ties in numerical order (1,2,3,4)

Reminder: At some point, you might add several variables to your propagation queue at once. Break ties by adding variables to your propagation queue in alphabetical order.

	Var assigned or de-queued	List all values eliminated from neighboring variables	Back track ?		Var assigned or de-queued	List all values eliminated from neighboring variables	Back track ?
ex	X	Y ≠ 3, 4 Z ≠ 3 (example)	☑	11			☐
1			☐	12			☐
2			☐	13			☐
3			☐	14			☐
4			☐	15			☐
5			☐	16			☐
6			☐	17			☐
7			☐	18			☐
8			☐	19			☐
9			☐	20			☐
10			☐	21			☐

2) Draw the complete search tree



LION	
HORNBILL	
ANTELOPE	
EVIL LION	
HYENA	
MEERKAT	
BOAR	

Question 3 (code)

Sudoku Boards as CSPs

Implement a CSP-based Sudoku solver that uses backtracking search, forward checking, and AC-3 to solve puzzles of varying difficulty.

To demonstrate that your CSP solver works, your program must solve the following four Sudoku boards. The results must be included in your report.

		4		3			5	
6		9	4					
		5	1			4	8	9
				6		9	3	
3			8		7			2
	2	6		4				
4	5	3			9	6		
					4	7		5
	9			5		2		

Figure 1: Easy board (`easy.txt`)

				3			4	
1		9	7					
			8	5	1		7	
		2	6		7	8	3	
9		6		1		2		7
	3	1	5		2	9		
	1		3	6	9			
					5	7		3
	9			7				

Figure 2: Medium board (`medium.txt`)

1		2		4				7
				8				
		9	5			3		4
			6		7	9		
5	4						2	6
		6	4		5			
7		8			3	4		
				1				
2				6		5		9

Figure 3: Hard board (`hard.txt`)

		1			7			
6			4			3		
				3			6	4
3	8			7	6			
							3	6
2	7			1	5			
				2			5	1
7			1			2		
		8			9			

Figure 4: Very hard board (`veryhard.txt`)

Input Format

Your program must read Sudoku boards from text files in the following format:

File contains exactly 9 lines

Each line contains exactly 9 digits (0–9)

0 represents an empty cell

Example (easy.txt):

004030050
609400000
005100489
000060930
300807002
026040000
453009600
000004705
090050200

Deliverables

1. Well-commented source code for a CSP solver based on backtracking search and AC-3, that is able to solve Sudoku boards.
2. Your program's solution for each of the four boards shown above.
3. The number of times your BACKTRACK function was called, and the number of times your BACKTRACK function returned failure, for each of the four boards shown above. Comment briefly on these numbers for each of the four boards.
4. Github link is must